

Окно *window*

Свойства окна. Переходы между окнами

Window

В WPF окно представлено классом `System.Windows.Window`. Это корневой элемент графического интерфейса, который может перемещаться, изменять размер, сворачиваться и т.д. Рассмотрим основные свойства, события и методы работы с окнами.

Основные свойства окна

Свойство	Описание
Title	Заголовок окна (отображается в строке заголовка).
Width / Height	Ширина и высота окна.
MinWidth / MinHeight	Минимальные размеры.
MaxWidth / MaxHeight	Максимальные размеры.
WindowState	Состояние окна: Normal, Maximized, Minimized.
WindowStyle	Стиль окна: SingleBorderWindow (обычное), None (без рамки), ThreeDBorderWindow, ToolWindow.
ResizeMode	Определяет, можно ли изменять размер: NoResize, CanResize, CanResizeWithGrip.
Top / Left	Координаты левого верхнего угла окна относительно экрана.
WindowStartupLocation	Позиция при открытии: Manual, CenterScreen, CenterOwner.
Owner	Ссылка на окно-владелец (используется для модальных и дочерних окон).
Icon	Иконка окна (рядом с заголовком).
ShowInTaskbar	Показывать ли кнопку окна на панели задач.
Topmost	Держать окно поверх всех остальных.
SizeToContent	Автоматически подгонять размер под содержимое: Manual, Width, Height, WidthAndHeight.
DialogResult	Результат модального диалога (обычно bool?), закрывает окно с этим значением.

Изменение свойств окна

Ссылка: 0

```
public MainWindow()
{
    InitializeComponent();

    // this - обращение к текущему объекту
    this.Title = "Главное окно";

    // окно будет запускаться в центре
    this.WindowStartupLocation = WindowStartupLocation.CenterScreen;

    // иконка не будет отображаться в панели задач
    this.ShowInTaskbar = false;
}
```

```
<Window x:Class="WpfAppNewWindow.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WpfAppNewWindow"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800" ResizeMode="NoResize">
```

Основные события окна

ContentRendered – возникает сразу же после первой визуализации окна.

Loaded – происходит когда окно полностью инициализировано и готово к взаимодействию.

Activated – возникает когда пользователь переключается на это окно, а также при первой загрузке окна.

Deactivated – возникает, когда пользователь переходит на другое окно, а также когда окно закрывается.

Closing – возникает при закрытии окна, позволяет отменить операцию закрытия.

Closed – возникает после закрытия окна.

Методы для управления окном

Show() – открывает окно в немодальном режиме. Поток выполнения продолжается.

ShowDialog() – открывает окно модально (блокирует родительское окно). Возвращает Nullable<bool> – DialogResult.

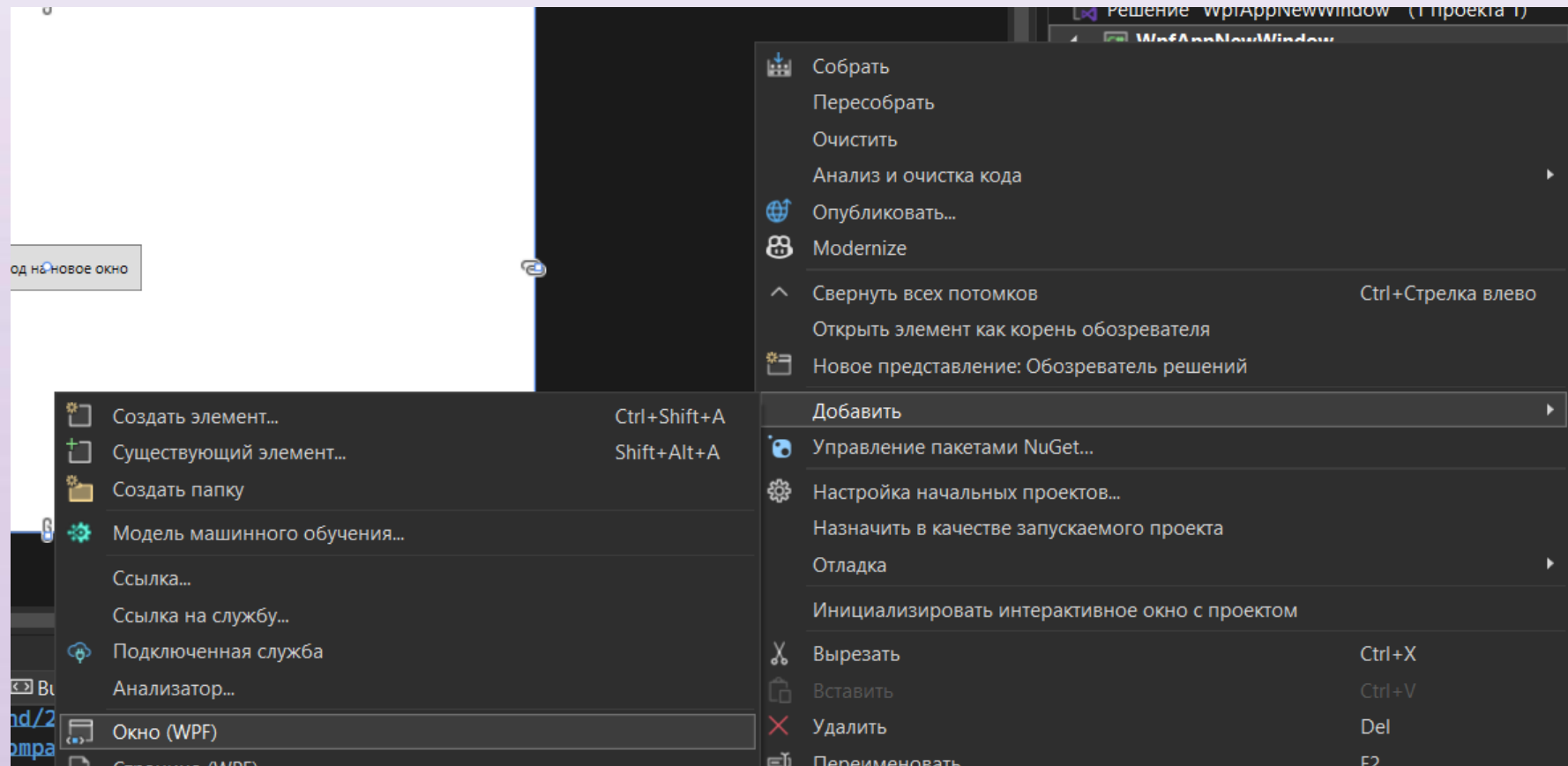
Close() – закрывает окно. Можно вызвать и внутри самого окна, и извне (если есть ссылка на экземпляр).

DragMove() – позволяет начать перетаскивание окна (полезно, если WindowStyle=None и нужно реализовать свою область захвата).

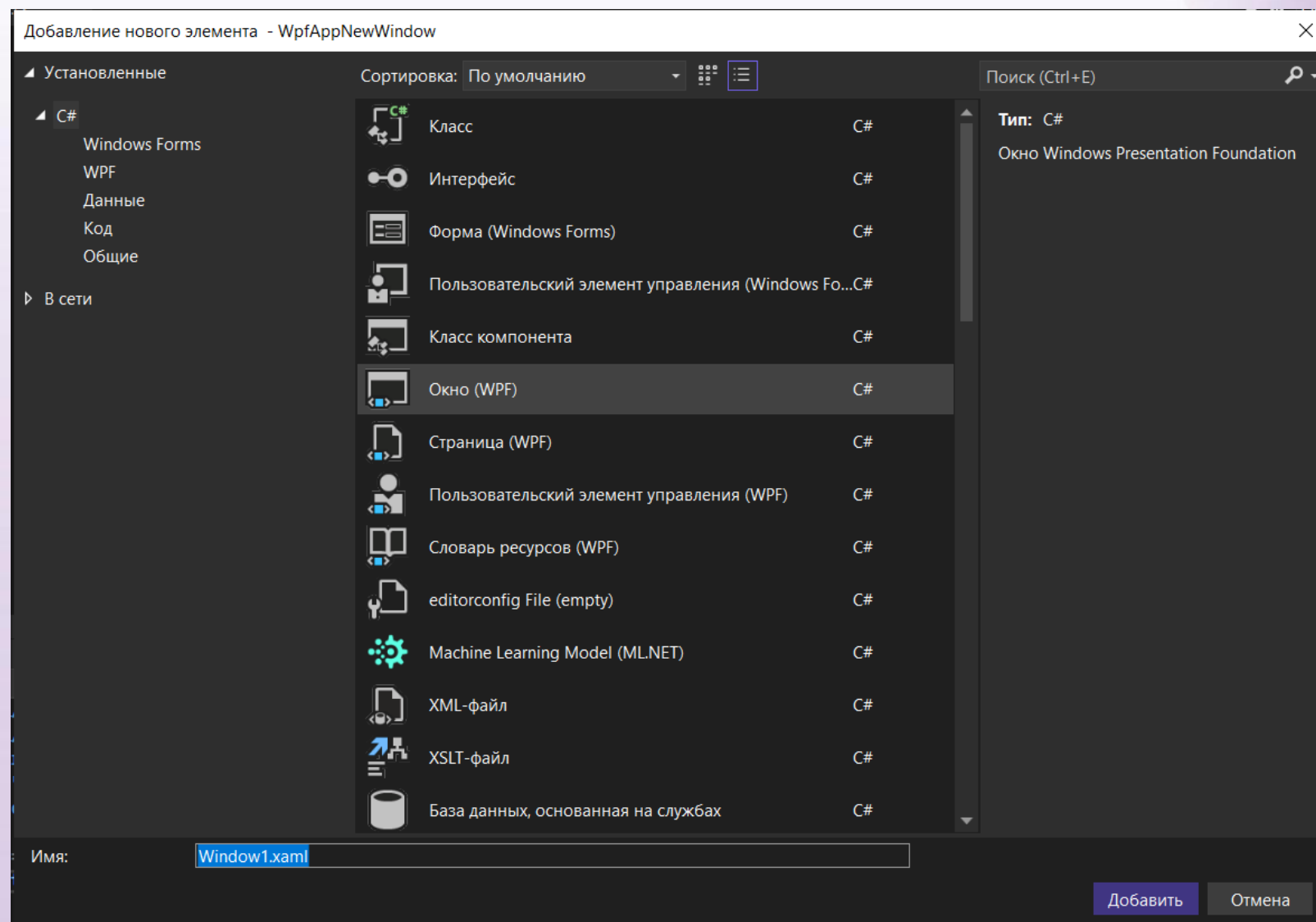
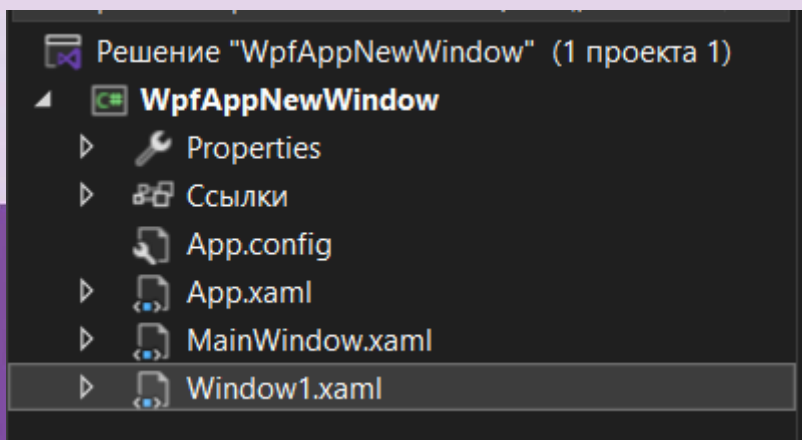
Activate() – делает окно активным (передает фокус).

Создание окна

Нажмите на проект ПКМ и выберите «Добавить» затем «Окно»



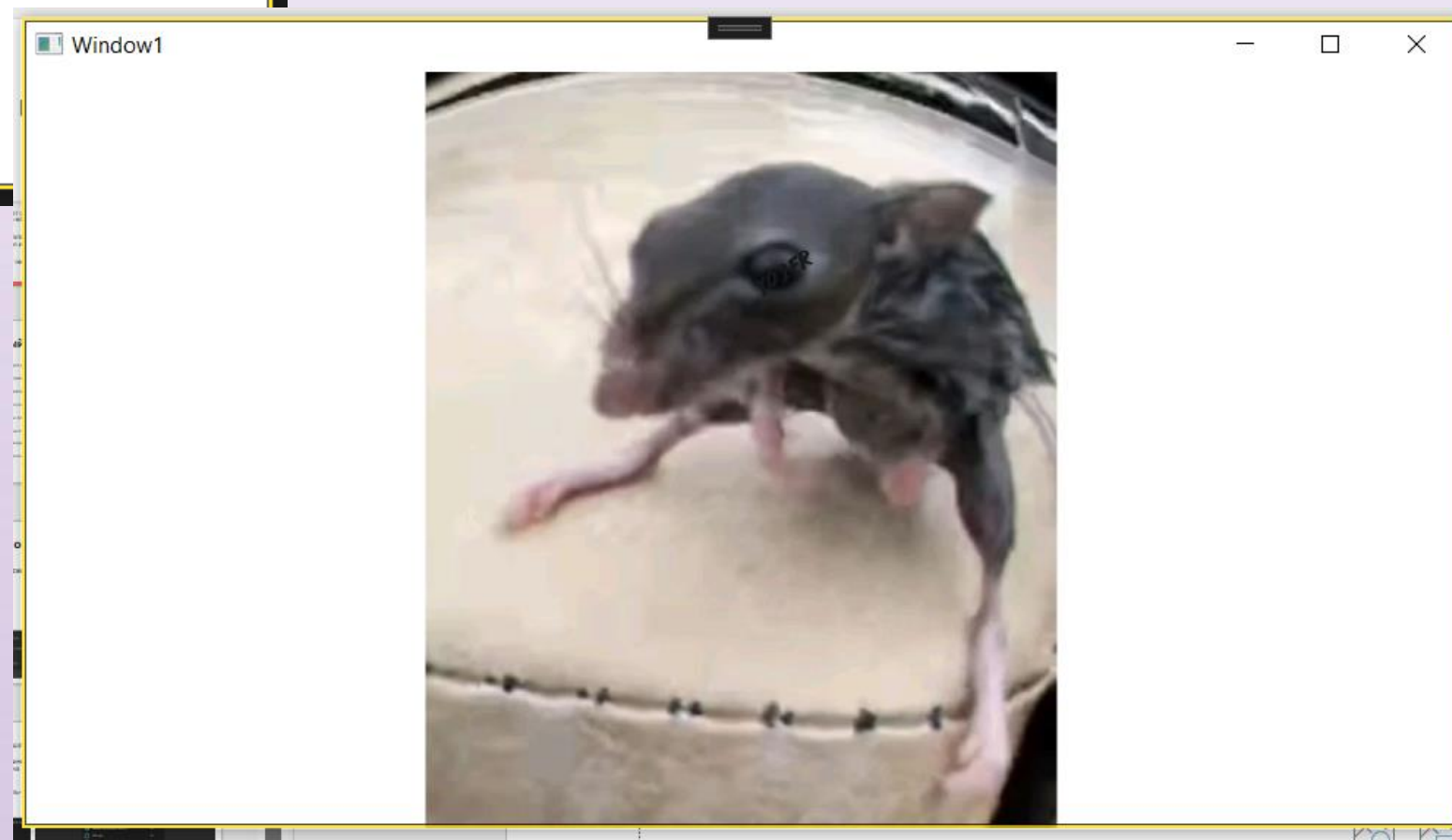
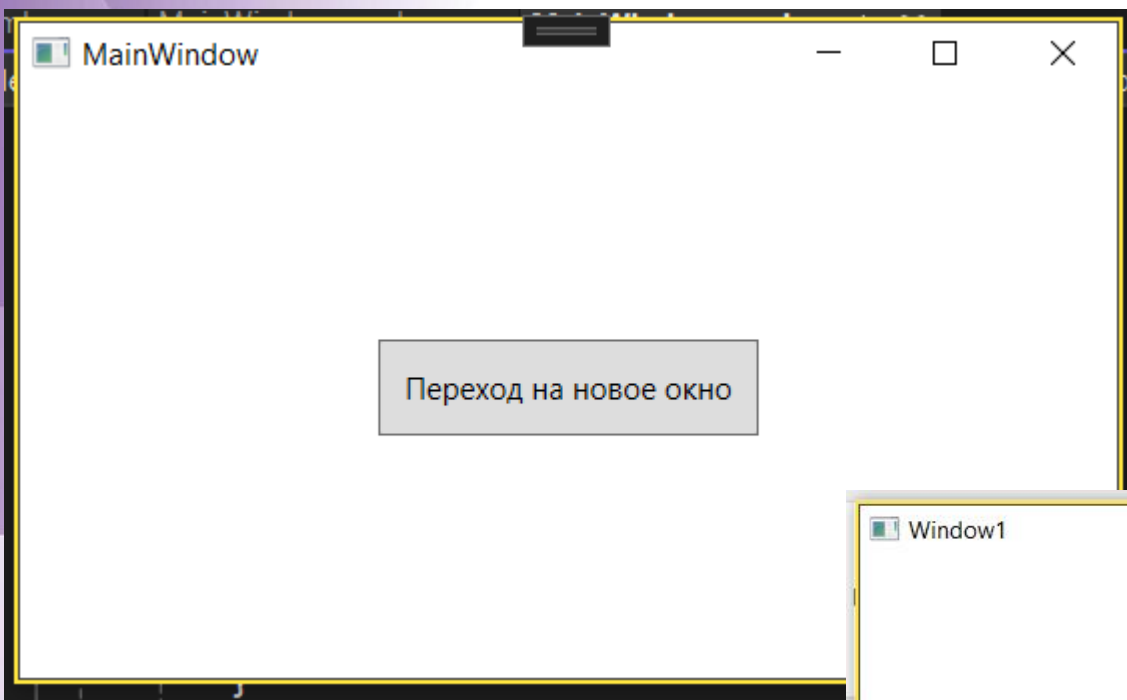
Предлагается создать окно Window1, вы можете поменять имя оно должно быть на английском и без пробелов. Далее нажмите «Добавить»



Открытие нового окна

```
Title="MainWindow" Height="450" Width="800">  
<Grid>  
    <Button x:Name="btnShowWindow" Content="Переход на новое окно" Padding="10"  
        HorizontalAlignment="Center" VerticalAlignment="Center" Click="btnShowWindow_Click"/>  
</Grid>
```

```
public partial class MainWindow : Window  
{  
    Ссылка: 0  
    public MainWindow()  
    {  
        InitializeComponent();  
    }  
  
    Ссылка: 1  
    private void btnShowWindow_Click(object sender, RoutedEventArgs e)  
    {  
        // создание экземпляра нового окна  
        Window1 newWindow = new Window1();  
        // открытие окна  
        newWindow.Show();  
        // закрытие текущего окна  
        this.Close();  
    }  
}
```



Типы диалоговых окон

- Модальное (запускается с помощью метода `ShowDialog()`) – доступ к родительскому окну запрещен пока открыто модальное окно.
- Немодальное (запускается с помощью метода `Show()`) – доступ к родительскому окну открыт.

```
private void Button_Click_1(object sender, RoutedEventArgs e)
{
    Window dialogWindow = new Window();
    if (dialogWindow.ShowDialog() == true)
    {
        // ...
    }
}
```

Взаимодействие окон

После открытия окна существуют независимо друг от друга. Можно закрыть главное окно, второе окно продолжит свою работу.

```
ССЫЛКА 1
private void btnShowWindow_Click(object sender, RoutedEventArgs e)
{
    // создание экземпляра нового окна
    Window1 newWindow = new Window1();

    newWindow.Owner = this;
    newWindow.WindowStartupLocation = WindowStartupLocation.CenterOwner;
    newWindow.Title = "Новое окно";

    // открытие окна
    newWindow.Show();
}
```

```
Title="Window1" Height="450" Width="800">
```

```
<Grid>
```

```
<Image Source="/загруженное.jpg" />
```

```
<Button Content="Изменить прошлое окно" Padding="10"  
        HorizontalAlignment="Center" VerticalAlignment="Center"  
        Background="{x:Null}" BorderBrush="#FFFF00DD"/>
```

```
</Grid>
```

Ссылка: 4

```
public partial class Window1 : Window
```

```
{
```

Ссылка: 1

```
public Window1()
```

```
{
```

```
    InitializeComponent();
```

```
}
```

Ссылка: 0

```
private void Button_Click(object sender, RoutedEventArgs e)
```

```
{
```

```
    (this.Owner as MainWindow).Title = "Окно владелец";
```

```
    this.Owner.Background = new SolidColorBrush(Colors.AliceBlue);
```

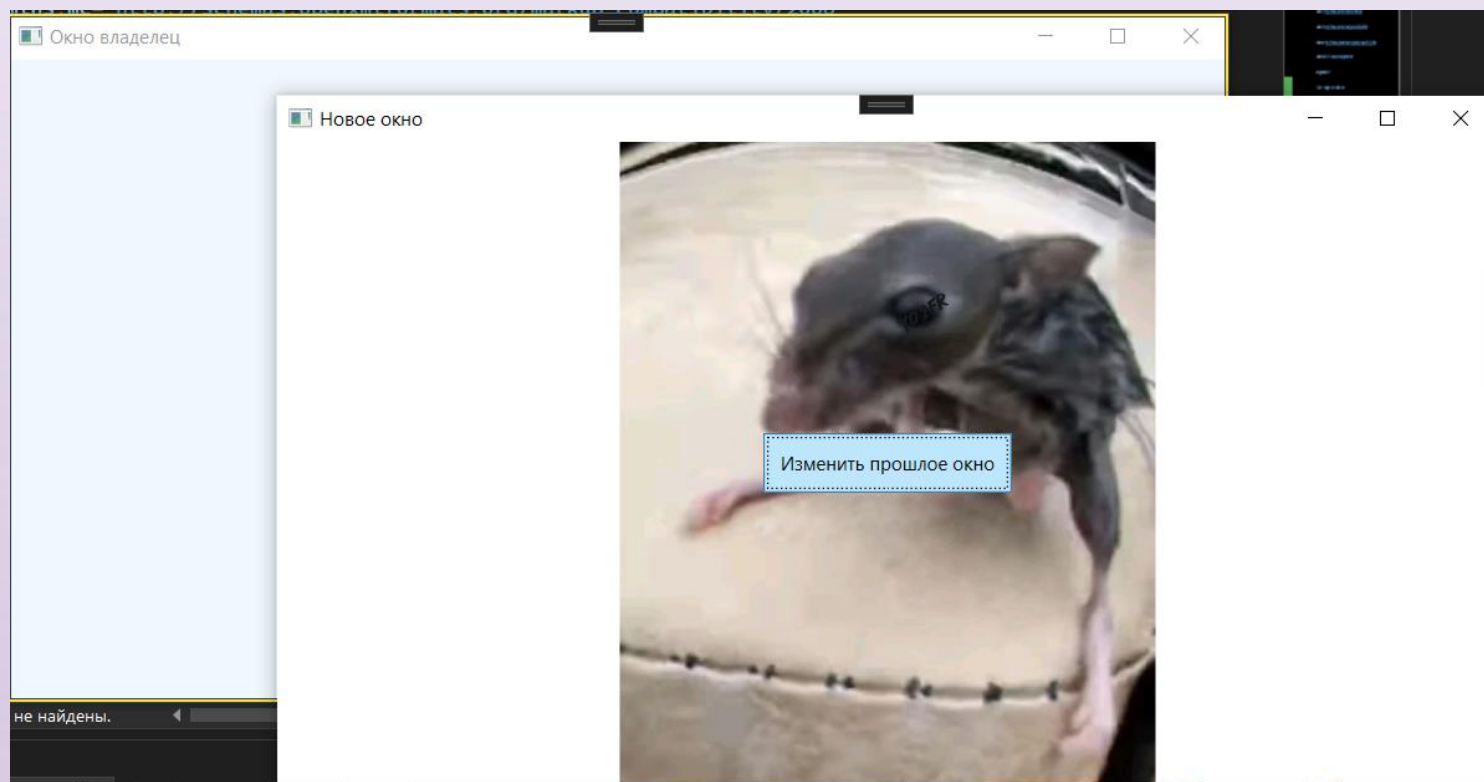
```
}
```

```
}
```

Для поддержки владения окна класс `Window` предлагает два свойства:

`Owner` – представляет собой ссылку, которая указывает на окно, владеющее текущим окном.

`OwnerWindows` – коллекция всех окон, которыми владеет текущее окно. Окна, обладающие окном-владельцем, всегда отображаются как немодальные



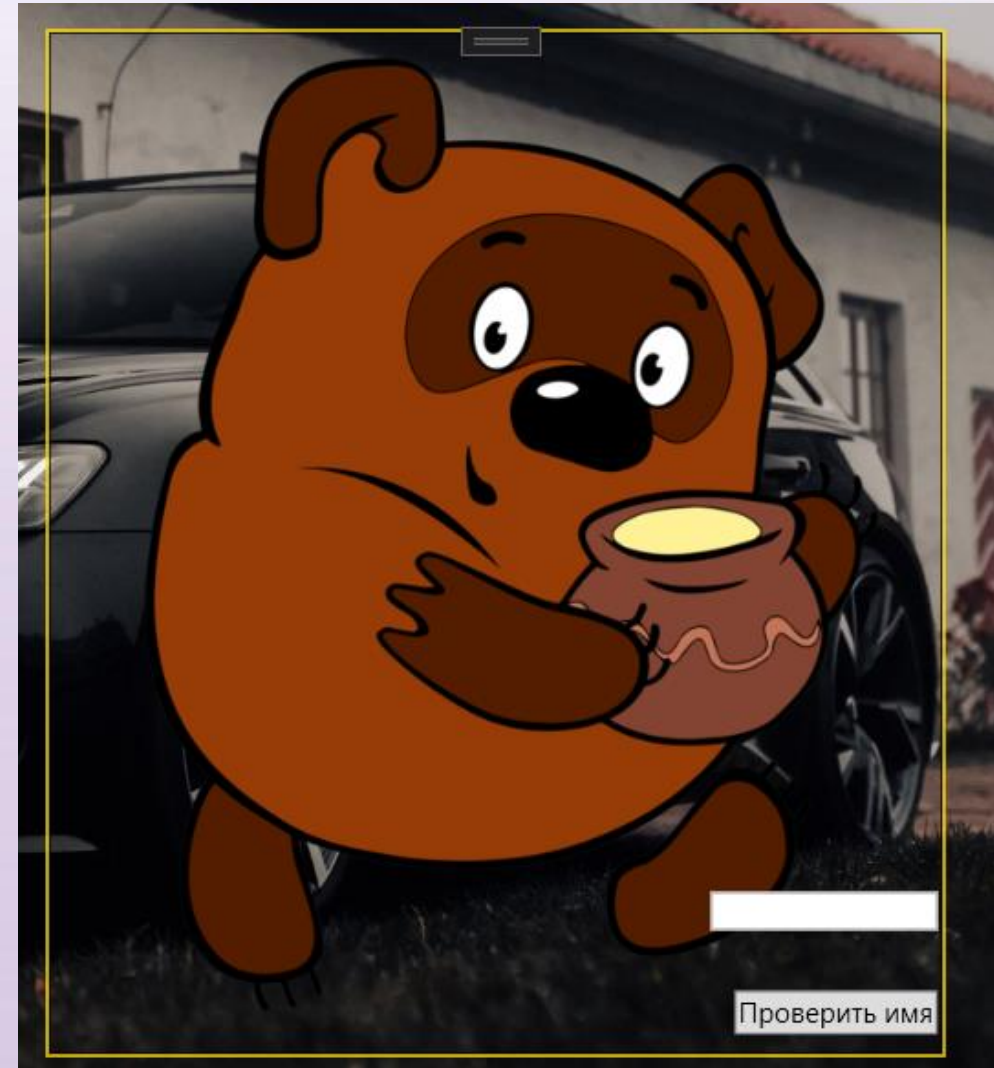
Задание - Нестандартные окна

Задать свойства `WindowStyle`, чтобы не было рамки и `AllowsTransparency` для прозрачности.

Для заднего фона выбрать png картинку.

Для движения окна:

```
Ссылка: 0  
private void Window_MouseDown(object sender, MouseButtonEventArgs e)  
{  
    this.DragMove();  
}
```



Задание - Нестандартные окна

После проверки имени, если имя верное, то открывается новое окно с картинкой и надписью «Проверка пройдена». Добавьте на это окно кнопку, которая будет менять png картинку у окна владельца и закрывать текущее окно.

Если имя не верное то открывается новое модальное окно с информацией, что имя не верное и кнопкой «Ок».

